Repairing Order-Dependent Flaky Tests with Command-Line Coding Agents: A Comparative Study of Gemini CLI and Claude Code

Milena Serbinova

Seminar: Software Quality
Advisor: Fabian Leinen
Technical University of Munich
milena.serbinova@tum.de

Abstract. Flaky tests pass or fail non-deterministically on unchanged code, undermining developer trust and continuous integration reliability. While recent frameworks leverage large language models for automated flaky test repair, most remain prompt-centric and lack the capacity for multi-step reasoning needed for complex projects. Command-line (CLI) coding agents, which can autonomously perform iterative tool-assisted actions, offer a lightweight alternative to existing approaches, though their effectiveness for flaky test repair remains unknown.
This study provides the first systematic evaluation of CLI coding agents, Gemini CLI and Claude Code, for the repair of 155 order-dependent flaky tests from real-world Python projects. Our results demonstrate that autonomous repair is viable, with both agents achieving overall success rates exceeding 50%. Notably, a significant gap exists between flakiness categories: agents repaired approximately 70% of victim tests but only 23% of brittle tests. This disparity highlights the difficulty agents face when independently identifying the root causes of flakiness without explicit pre-validation information.
The results show that CLI coding agents are a powerful support tool for developers rather than a universal solution for test flakiness. Efficiency analysis reveals that 67% of successful fixes occur on the first attempt, with success rates diminishing sharply in subsequent iterations. These findings suggest that future research approaches should focus on providing the agents with richer execution context rather than increasing iteration limits. Finally, while repair success was comparable, Claude Code operated over 40% faster than Gemini CLI, offering higher throughput for real-world pipelines.

Keywords: Flaky test repair · CLI coding agents · order-dependent flaky tests · autonomous repair

1 Introduction

Software tests are commonly referred to as *flaky* when they non-deterministically pass or fail on identical code without any corresponding changes to the test or production environment. Prior studies have identified multiple root causes of flakiness [8][24], with test-order dependence being one of the most prevalent [12][28]. Order-dependent flaky tests pass or fail based on their execution order. Unlike other forms of flakiness, these tests can be reliably detected and reproduced by systematically reordering the test suite [10][41]. This predictability allows for the controlled validation of repair strategies.

Test flakiness is widely recognized as a significant challenge that introduces misleading signals into continuous integration systems and undermines trust in automated test suites [11][18][29]. While numerous techniques have been proposed to mitigate these negative impacts through automated repair, early frameworks targeting order-dependent flakiness, such

as iFixFlakies [32], iPFlakies [38], and ODRepair [20], are limited by their reliance on a constrained set of predefined repair strategies. Consequently, these approaches often fail to handle unexpected test interactions and lack the flexibility required to synthesize semantically rich repairs tailored to complex test logic [6].

To address these limitations, recent research has introduced specialized frameworks that leverage large language models (LLMs) to analyze execution traces and generate repairs [5][9][19]. Despite the promising results of the LLM-based frameworks, they remain largely prompt-centric and single-shot in nature, relying on static context snapshots that prevent them from iteratively exploring or localizing complex dependencies between tests.

In parallel, the software development industry has witnessed widespread adoption of coding agents. Unlike traditional LLMs that produce code snippets from single prompts, they operate within dynamic development environments and perform iterative, tool-assisted actions to achieve complex goals [36]. Recent evidence shows that a majority of software developers, across both industrial and academic settings, now regularly use coding agents as part of their daily development practice [22][31].

A prominent example of this trend is command-line (CLI) coding agents such as Claude Code[1] and Gemini CLI[2], which can interact with codebases through shell commands, autonomously analyze code, apply changes, and run applications. The effectiveness of these agents depends not only on the underlying LLM but also on their architectural differences in how they analyze tasks and orchestrate actions [3][37]. This makes an evaluation on real-world development workflows essential to assess their feasibility and architectural influence.

Due to their minimal setup requirements and greater autonomy, CLI coding agents offer a lightweight alternative to existing LLM-based flakiness repair frameworks. By interacting directly with the command line and development environment, such agents reduce orchestration complexity and lower the barrier to deployment in real-world settings. However, it remains unclear whether this architectural simplicity translates into effective handling of challenging, non-deterministic, multi-step tasks such as flaky test repair. An evaluation in this setting is essential to determine if autonomous repair is feasible and to compare how different agent architectures perform on complex software engineering tasks.

To address this gap, we systematically evaluate the repair of flaky real-world tests using two widely used CLI coding agents, Claude Code and Gemini CLI. We apply up to five repair attempts per agent to 155 order-dependent flaky tests under identical conditions from the International Dataset of Flaky Tests [16], focusing on Python projects using pytest. The controlled comparison enables us to assess both the feasibility of autonomous repair using coding agents and the extent to which different agent architectures influence repair success.

Our evaluation reveals that both Gemini CLI and Claude Code achieve slight majority success rates of approximately 51–54% in repairing order-dependent flaky tests, indicating that autonomous flaky test repair using CLI coding agents is practically viable for a meaningful subset of cases. However, approximately 40% of the tests remained flaky despite multiple attempts, and most successful repairs occur on the first attempt. This pattern suggests that repair success depends more on initial problem comprehension than on iterative refinement and that current agent architectures may lack the sophisticated failure analysis mechanisms needed to effectively learn from unsuccessful repair attempts when dealing with non-deterministic test behavior.

[1] https://claude.com/product/claude-code
[2] https://geminicli.com/

2 Background

2.1 Flaky Test Classification

Test flakiness has been extensively studied in software engineering research, leading to multiple taxonomies of flaky test patterns. Lam et al. divide flaky test cases into two main categories: order-dependent (OD) tests, which deterministically pass or fail depending on the execution order of tests, and non-order-dependent (NOD) tests [18]. Building on this foundation, Shi et al. further refine the classification of OD tests by distinguishing subcategories based on the outcome when a test is executed in isolation [33]. Specifically, they identify victim tests (OD-Vic), which pass when run in isolation but fail when executed after certain other tests, and brittle tests (OD-Brit), which fail when run in isolation but pass when executed alongside other tests. In addition to behavior-based taxonomies, Luo et al. introduce a root-cause-oriented classification that includes order dependency as one category among several, such as timing issues, concurrency, and asynchronous waits [24].

2.2 Flaky Test Detection

The research community has proposed a range of approaches aimed at improving flaky test detection. In particular, iDFlakies [18] and Shaker [34] detect flakiness by executing tests under different orders and environments. Such rerun-based methods represent the most common industry practice [2][43]. When a test initially fails, practitioners rerun it multiple times without modifying either the code under test or the test itself. If the results differ across these runs, the test is classified as flaky [24]. However, this approach consumes a substantial amount of computing and storage resources and increases developer feedback loop time [2].

This has motivated recent work which has explored alternative detection techniques based on static or lightweight features that do not require repeated test execution [15][26][42]. Several datasets support research in this area, including the International Dataset of Flaky Tests (IDoFT) [16] and the FlakyCat dataset [1].

3 Related Work

3.1 LLM-Based Approaches to Flaky Test Repair

Recent advances in LLMs have enabled a range of automated approaches for flaky test repair, with several systems demonstrating promising results. FlakyFix pioneered a two-stage pipeline that combines pretrained code models, CodeBERT and UniXcoder, for fix category prediction with GPT-3.5 for repair generation. The authors report that 51–83% of generated repairs were expected to pass the test suite [9]. FlakyDoctor introduced a neuro-symbolic approach that integrates LLMs with program analysis techniques, achieving success rates of 57–59% on their benchmark [5]. Most recently, FlakyGuard has addressed the *context problem*, where insufficient context may omit relevant production code while excessive context can overwhelm the LLM with irrelevant information [19]. FlakyGuard uses LLM-guided exploration of dynamic call graphs and reports a repair rate of 47.6%, with 51.8% developer acceptance rate in deployment, outperforming FlakyDoctor by 22%.

While these systems represent substantial progress, they share limitations that affect their practical applicability. In particular, they remain largely prompt-centric and rely on limited validation cycles, which makes them prone to incomplete root-cause localization and repairs that overfit to a single observed failing order rather than resolving the underlying state interference. Coding agents represent a natural next step by shifting from static prompt orchestration to autonomous, multi-step reasoning and interaction with the codebase.

3.2 Coding Agents for Software Engineering Tasks

While standalone LLMs perform well on isolated software engineering tasks such as code generation, testing, and repair, empirical studies show that LLM-based agents consistently achieve higher performance across code generation benchmarks of varying complexity, as well as in testing, debugging, and static analysis tasks [23]. Li et al. evaluated over 400,000 agent-authored pull requests and identified a trade-off between dramatically higher throughput and faster turnaround times on the one hand, but lower acceptance rates compared to human contributions on the other [21]. Their study also reports notable variation in acceptance rates across different pull request types. For documentation tasks, Claude Code achieved an acceptance rate of 85.7%, outperforming the human baseline of 76.5%, whereas across other categories it achieved an average acceptance rate of 52%, compared to 76.8% for humans. These findings highlight the importance of task sensitivity in agent-based evaluations, as agent performance can vary substantially across different categories of software engineering activities.

Given the central role of testing in the software development lifecycle, several studies have focused specifically on evaluating coding agents in test-related scenarios. SWT-Bench, a benchmark for issue-reproducing test generation on real-world Python repositories, shows that code agents outperform LLM-only and specialized test-generation systems at reproducing real issues [27]. At the same time, it finds that agent performance on test generation and code repair is only weakly correlated, indicating that they constitute distinct tasks. One of the first attempts to use agents to perform bug fixes, based on 178 bugs drawn from Google's issue tracking system, evaluates Gemini as an independent agent for program repair [30]. The authors report that Gemini 1.5 Pro, which was a flagship LLM in February 2024, successfully patched 43% of machine-reported bugs and 17.9% of human-reported bugs.

Techniques that are effective for repairing deterministic bugs do not necessarily transfer to the probabilistic nature of flaky test failures. This creates an opportunity to investigate how coding agents can be adapted or specialized for this distinct class of software quality issues. In addition, most existing approaches to LLM-based software repair rely on dedicated systems, such as RepairAgent [4], LANTERN [25], and PReMM [40]. This points to a gap in the evaluation of raw agent capabilities as delivered by end-to-end tools.

3.3 Comparative Evaluations of Coding Agents

Given the rapid evolution of coding agents, the evaluation of code generation increasingly relies on standardized benchmark datasets. Recently proposed benchmarks aim to better reflect realistic software development tasks, with a particular focus on the interaction, planning, and execution capabilities of agents operating in complex environments [7].

Modern benchmarks such as LiveBench explicitly measure agentic coding behavior using metrics like the *agentic coding average*, which assesses a model's ability to understand code, interact with tools, and complete multi-step coding tasks in dynamic settings [39]. SWE-bench evaluates the ability of systems to resolve real-world GitHub issues [14] and includes a dedicated bash-only setting to assess LLMs operating in a minimal command-line environment. Despite these advances, recent studies point to persistent challenges in benchmarking LLMs for software engineering, including limited realism, potential construction bias, and incomplete coverage of real-world development tasks [13].

4 Approach

Our goal is to evaluate whether widely-used CLI coding agents can autonomously repair flaky tests without requiring fine-tuning or custom integration, and compare the effectiveness across different agent architectures. To achieve this, we design a system that provides minimal yet

sufficient context to the agent: test metadata, Pre-Validation results, and failure information. The agent receives up to five attempts per test, during which it can iteratively diagnose errors, apply repairs, and validate them.

Our approach is built on three core principles. First, we employ category-aware validation strategies that align with the distinct forms of order-dependent flakiness (OD-Vic and OD-Brit), ensuring that repairs address the specific failure modes observed during Pre-Validation. Second, we enable genuine learning across attempts by maintaining conversation context while automatically resetting repository state after each failed attempt, allowing the agent to refine its strategy without operating on corrupted code. Third, we constrain modifications to test code only, prioritizing safety and aligning with empirical evidence that most flaky test fixes can be achieved without altering production code [8][24].

4.1 System Architecture

Our system implements a fully autonomous pipeline that takes a flaky test specification as input and allows a CLI coding agent to iteratively analyze, repair, and validate the test until it either produces a stable fix or exhausts a predefined attempt budget. Figure 1 illustrates the main components that work together to achieve this goal.

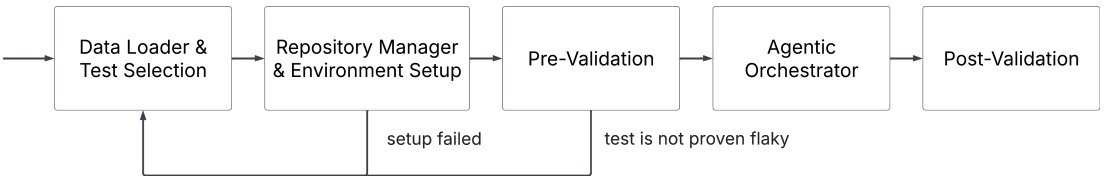


Fig. 1. System workflow

Data Loader and Test Selection Our study was conducted using the IDoFT dataset. At the start of the evaluation process, we created a local copy of the dataset and extended it by adding columns to store metadata and results.

The Data Loader is responsible for selecting tests from the dataset and applying filtering strategies to ensure reproducibility and fair comparison across models. Tests are randomly sampled from the set of available candidates that satisfy the following criteria: (1) the test belongs to the OD-Vic or OD-Brit category, excluding NOD tests as discussed in Section 4.2; (2) the test has not been previously attempted; and (3) the repository size does not exceed 500MB to ensure reasonable clone times.

Repository Manager and Environment Setup The Repository Manager automates the setup process required to execute tests in a reproducible environment. It clones the target repository at a specified commit and creates an isolated virtual environment with all necessary dependencies. To improve performance across repeated evaluations, the component employs a two-tier caching strategy. Repositories are cached by commit hash, allowing reuse when multiple tests are evaluated on the same snapshot. Virtual environments are cached per repository, eliminating repeated dependency installation when processing multiple tests from the same codebase. These mechanisms significantly reduce setup overhead during large-scale evaluations.

Before validation begins, the component verifies that the target test can be successfully discovered by the testing framework. This early check, based on the framework's test collection mechanism, helps avoid wasted computation on tests that may have been renamed, relocated, or removed at the specified commit.

Pre-Validation Before attempting a repair, the system verifies that a test is flaky within the current environment. It executes the target test in isolation five times and then runs it as part of the full suite across 100 randomized execution orders, each defined by a unique seed.

Flakiness validation differs by test category to reflect distinct forms of order-dependent flakiness. For OD-Vic tests, following Lam et al. [18], the system confirms flakiness if the test consistently passes in isolation but fails in at least one suite ordering. Conversely, an OD-Brit test is confirmed as flaky if it fails in isolation but passes in at least one suite ordering, which demonstrates a dependence on setup from other tests. Regardless of the category, the system records all failure-inducing seeds and provides them to the CLI coding agent as reproducible failure cases. These same seeds are reused in the Post-Validation step to ensure that the proposed repairs specifically address the observed failure modes.

Agentic Orchestrator The Agentic Orchestrator is the core component of the system, managing the interaction with the CLI coding agent across up to five iterative repair attempts. It executes an autonomous loop that allows the agent to incorporate failure feedback and refine its repair strategy over time.

Listing 1.1. Structured context passed to the coding agent

```
context = {
    "test_info": {
        "test_name": "...",
        "category": "OD-Vic|OD-Brit",
        "test_file": "path/to/test.py"
    },
    "repo_path": "/path/to/repo",
    "python_environment": {
        "pytest_command": "pytest",
        "note": "Virtual environment is activated"
    },
    "pre_validation_results": { ... },
    "failed_sequences_info": "Formatted description of failures",
    "attempts_so_far": [ ... ]  # Updated after each attempt
}
```

For each attempt, the Orchestrator constructs a structured context dictionary, as illustrated in Listing 1.1, providing the agent with the necessary information to diagnose and fix the test. This context includes Pre-Validation results with detailed data regarding which execution orders failed, formatted as reproducible commands with specific seeds. Additionally, the agent receives the test category, the repository path, and the results of any previous repair attempts to prevent the repetition of failed strategies.

Algorithm 1 facilitates iterative reasoning across up to five repair attempts. When an attempt fails, the Git repository is restored to its clean baseline as described in Section 4.4. However, the conversation context with the LLM persists, allowing the agent to analyze why previous approaches failed and explore alternative strategies.

Post-Validation If the Agentic Orchestrator reports success, the Post-Validation component performs a final verification to ensure the fix is robust. This component utilizes the same category-aware validation strategies as Pre-Validation but with a critical enhancement: it reuses the specific failure-inducing seeds identified during the initial detection phase.

Post-Validation executes the test in isolation five times and then within the full test suite using the exact seeds that previously caused failures. To ensure a stable fix, the test must pass all executions regardless of its category. This verification strategy confirms that the proposed

Algorithm 1 Main orchestration loop for flaky test repair

```
for attempt = 1 to MaxAttempts do
    Build or update context using results from previous attempts
    Generate repair prompt with full context
    Invoke coding agent via terminal interface

    After agent completion:
    if no file changes detected then
        Mark attempt as failed
        Continue to next iteration
    else
        Run quick validation (10 test orderings)
        if quick validation fails then
            if attempt < MaxAttempts then
                Restore repository state
                Continue to next iteration
            else
                Terminate
            end if
        else
            Run full validation (100 test orderings)
            if full validation passes then
                Terminate with success
            else
                if attempt < MaxAttempts then
                    Restore repository state
                    Continue to next iteration
                else
                    Terminate
                end if
            end if
        end if
    end if
end for
```

repairs address the specific observed failure modes rather than passing coincidentally due to fortunate seed selection.

4.2 Flaky Test Categories

Our approach focuses exclusively on two categories of order-dependent flakiness: OD-Vic and OD-Brit. We exclude NOD flaky tests from the evaluation for several reasons:

- **Detection Reliability**: Order-dependent flakiness can be reliably detected and reproduced through test reordering, with failure-inducing seeds enabling consistent replication [18]. In contrast, NOD flakiness often depends on timing, concurrency, or external system behavior, which makes failures sporadic and difficult to reproduce within a bounded number of validation runs [12].
- **Validation Complexity**: Validating repairs for NOD flaky tests requires substantially more executions to achieve statistically meaningful confidence [12]. As there are no deterministic failure-inducing orders, it is difficult to distinguish genuine fixes from chance success. This increases computational overhead and uncertainty, complicating fair and reproducible comparisons between agents.

4.3 Agentic Orchestration Prompt

To ensure consistent task framing, we provide a structured prompt that defines the agent's role, repair objectives, operational constraints, and iteration protocol. The prompt is category-aware but repository-agnostic, consisting of:

- **Role Definition**: Establishes the agent as an autonomous flaky test repair system with file editing and test execution capability.
- **Success Criterion**: Requires tests to pass consistently in isolation and across all suite orderings.
- **Category Guidance**: Provides OD-Vic and OD-Brit definitions, validation strategies, and common fix patterns.
- **Iteration Protocol**: Structures each attempt as analysis → hypothesis → implementation → validation → reporting, with a persistent reasoning file to avoid repeating mistakes.
- **Operational Constraints**: Limits tool calls (≤ 15) and modifications (test code only).
- **Context Data**: Supplies repository path, test category, failure-inducing seeds, and previous attempt results.

4.4 Key Design Decisions

Maximum Attempt Limits We impose a limit of five repair attempts per test, a choice informed by pilot evaluation and validated by our empirical results (Section 5.3). This limit balances repair thoroughness with computational efficiency.

Analysis of our pilot runs showed that successful fixes rarely occurred beyond the third attempt when earlier attempts had failed. Our full evaluation confirms this pattern: 88.1% and 75.0% of successful fixes for Gemini CLI and Claude Code, respectively, occur within the first two attempts (Table 3), with success rates declining sharply thereafter.

Five attempts provide sufficient opportunity for: (1) an initial hypothesis-driven fix, (2) refinement based on validation feedback, (3) exploration of alternative repair strategies, and (4–5) final debugging of edge cases. This constraint also serves a practical function in deployment: each attempt incurs LLM API costs and test execution time, and bounding iterations prevents runaway resource consumption on tests that exceed current model capabilities.

Automated Repository State Reset After each failed repair attempt, the system automatically restores the repository to its original state using version control commands, rather than relying on the coding agent to undo changes via terminal commands. This ensures each attempt begins from a clean baseline, while the conversation history and persistent reasoning file maintain continuity across iterations.

This design choice offers several key advantages. It reduces cognitive burden and preserves the tool call budget. Manual cleanup would require the agent to execute multiple version control commands, verify file states, and confirm successful restoration—consuming tool calls that could otherwise support diagnosis, implementation, and validation activities. At the same time, it eliminates fragile inter-attempt dependencies. If agent-driven state restoration were to fail even once, subsequent iterations would operate on a corrupted codebase, potentially causing cascading failures across all remaining attempts. Automatic orchestrator-level reset guarantees a consistent starting state regardless of agent actions.

Focus on Test Code Modifications The orchestration prompt instructs the coding agent to focus repairs exclusively on the test code, avoiding modifications to production source files or infrastructure configuration. This constraint is well-supported by prior research: empirical studies indicate that the majority of flaky test fixes can be successfully achieved through test-only modifications [8][17][24].

This design choice serves multiple purposes. First, it prioritizes safety: modifying production code to accommodate flaky tests risks introducing functional bugs into the system under test. Second, it aligns with empirical evidence showing that OD flakiness is predominantly caused by flaws in test design, such as polluted shared state, missing cleanup, or insufficient initialization, rather than defects in the production code itself [33]. Third, constraining the modification scope helps the agent maintain focus on a manageable context window, potentially improving fix quality by reducing the cognitive load associated with navigating large codebases [19].

In our evaluation, agents predominantly respect this constraint, with the vast majority of proposed fixes targeting the specific test file under repair. Occasional modifications to test utilities or fixtures remain within the testing codebase and do not affect production behavior.

5 Evaluation

5.1 RQ1: How effective are CLI coding agents at repairing order-dependent flaky tests?

Motivation The central question of our evaluation is whether widely-used CLI coding agents can autonomously repair flaky tests without human intervention. By comparing Gemini CLI and Claude Code on the same benchmark, we assess both the feasibility of autonomous repair and the extent to which agent architecture influences success rates.

Results We evaluated both agents on 155 order-dependent flaky tests from IDoFT. Both Gemini CLI and Claude Code demonstrate that autonomous flaky test repair is feasible, successfully repairing a substantial portion of the randomly selected tests. However, more than 40% remain flaky after attempts by either agent, indicating considerable room for improvement.

Table 1. Overall success rates on 155 randomly selected tests

Outcome	Gemini CLI		Claude Code	
	Absolute	**Relative (%)**	**Absolute**	**Relative (%)**
Success	84	54.2	80	51.6
Failure	71	45.8	75	48.4

Table 1 shows that Gemini CLI achieved a success rate of 54.2% compared to 51.6% for Claude Code, a difference of 2.6 percentage points. While Gemini performs slightly better in absolute terms, this gap is not statistically significant (two-proportion z-test, $p > 0.05$).

Both agents repaired 73 tests (47.1%) representing 86.9% of Gemini CLI's fixes and 91.3% of Claude Code's fixes. Gemini CLI exclusively fixed 11 tests (7.1% of total, 13.1% of its successes) that Claude Code failed after five attempts, while Claude Code uniquely fixed 7 tests (4.5% of total, 8.8% of its successes), as illustrated in Figure 2. These 18 non-overlapping fixes (11.6% of total) demonstrate complementary capabilities between the two agents. Nevertheless, 64 tests (41.3%) remained flaky after five attempts by both agents.

Discussion The 51–54% success rate indicates that autonomous flaky test repair using CLI coding agents is practically viable for a meaningful subset of tests, achieving majority success while also highlighting that a substantial portion of flaky tests remain beyond current agent capabilities. The high overlap (73 tests fixed by both, representing 87–91% of each agent's fixes) suggests that certain flakiness patterns are reliably addressable by CLI coding agents regardless of the underlying model and agent architecture.

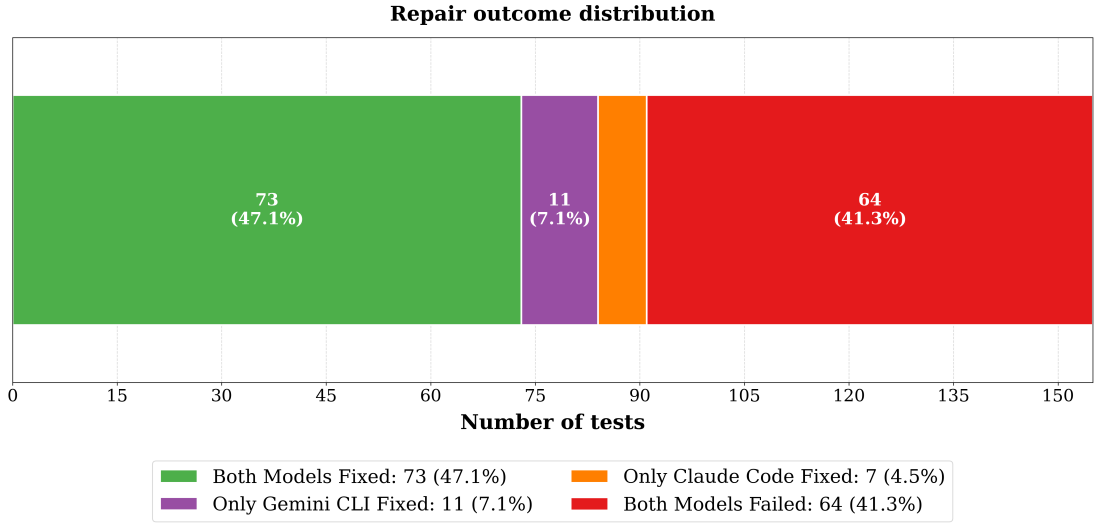


Fig. 2. Repair outcome distribution across both agents

The 18 non-overlapping fixes (11.6% of total) demonstrate complementary capabilities, suggesting that different agents may excel at different flakiness patterns. This opens opportunities for ensemble approaches that combine multiple agents. The 64 tests (41.3%) that remained flaky after attempts by both agents represent challenging cases that likely involve complex cross-test dependencies or external system interactions that exceed existing agentic capabilities.

5.2 RQ2: How effective are CLI coding agents at repairing different types of order-dependent flaky tests?

Motivation Understanding which types of flakiness patterns are amenable to automated repair helps identify the boundaries of CLI coding agent capabilities and can inform future prompt engineering and tool development.

Table 2. Success rates by flakiness category (OD-Vic vs. OD-Brit)

Category	Total	Gemini CLI	Claude Code	Δ (pp)
OD-Vic	100	72 (72.0%)	67 (67.0%)	5.0pp
OD-Brit	55	12 (21.8%)	13 (23.6%)	-1.8pp
Overall	**155**	**84 (54.2%)**	**80 (51.6%)**	**2.6pp**

Results The full IDoFT is skewed toward OD-Vic tests (804 OD-Vic, 319 OD-Brit). Our randomly selected dataset reflects this distribution with 100 OD-Vic and 55 OD-Brit tests.

Table 2 reveals a striking performance difference between the two flakiness categories. Both agents achieve substantially higher success rates on OD-Vic tests (72.0% for Gemini CLI, 67.0% for Claude Code) compared to OD-Brit tests (21.8% for Gemini CLI, 23.6% for Claude Code). Thus, OD-Vic tests are approximately 3 times more likely to be repaired than OD-Brit tests. The performance gap between agents remains relatively consistent across categories (5.0 percentage

points for OD-Vic, 1.8 percentage points for OD-Brit). Neither category shows statistically significant differences between agents (two-proportion z-test, $p > 0.05$).

Discussion The dramatic performance gap between OD-Vic (67–72% success) and OD-Brit (22–24% success) tests highlights a fundamental difference in the diagnostic information required for a successful repair. For OD-Vic tests, the failure-inducing seeds provided by the Orchestrator help the CLI coding agent to identify the specific test that must be isolated or cleaned up to prevent the state pollution that triggers the failure. [33].

However, repairing an OD-Brit test requires identifying the tests that provide the necessary setup for the flaky test to pass [33]. Since the system currently focuses on providing failure-inducing sequences, CLI coding agents lack direct evidence of which specific test orders resulted in a successful run. This absence of passing-sequence data forces the agent to search for dependencies across the codebase rather than analyzing a known successful trace, which transforms a localization task into an expensive search and significantly increases repair difficulty.

The performance gap between agents remains small across both categories (5.0pp for OD-Vic, 1.8pp for OD-Brit), suggesting that the relative performance of agents is independent of flakiness type, but both agents struggle equally with lack of explicit information about successful execution paths, which is necessary to identify the missing state requirements for brittle tests.

5.3 RQ3: How do iteration count and execution time affect CLI coding agent success?

Motivation Repair efficiency, measured by both iteration count and execution time, directly impacts deployment feasibility and cost. Understanding how quickly agents converge to solutions reveals whether the iterative approach enables genuine learning or whether agents either succeed immediately or fail to improve.

Table 3. Attempt distribution for successful runs

Attempts	Gemini CLI		Claude Code	
	Count	%	Count	%
1	56	66.7	53	66.3
2	18	21.4	7	8.8
3	5	6.0	9	11.3
4	3	3.6	8	10.0
5	2	2.4	3	3.8
Total	84	100	80	100
Mean	1.54	-	1.76	-
Median	1.0	-	1.0	-

Results Both agents achieved the majority of their successes on the first attempt, 66.7% for Gemini CLI and 66.3% for Claude Code, as shown in Table 3. The mean attempts for repairs were 1.54 for Gemini CLI and 1.76 for Claude Code, with both agents having a median of 1.0. A smaller fraction of fixes occurred in the second attempt (21.4% for Gemini CLI, 8.8% for Claude Code), with success rates continuing to decline in subsequent attempts. Attempts 3–5 collectively accounted for 11.9% of Gemini CLI's fixes and 25.0% of Claude Code's fixes.

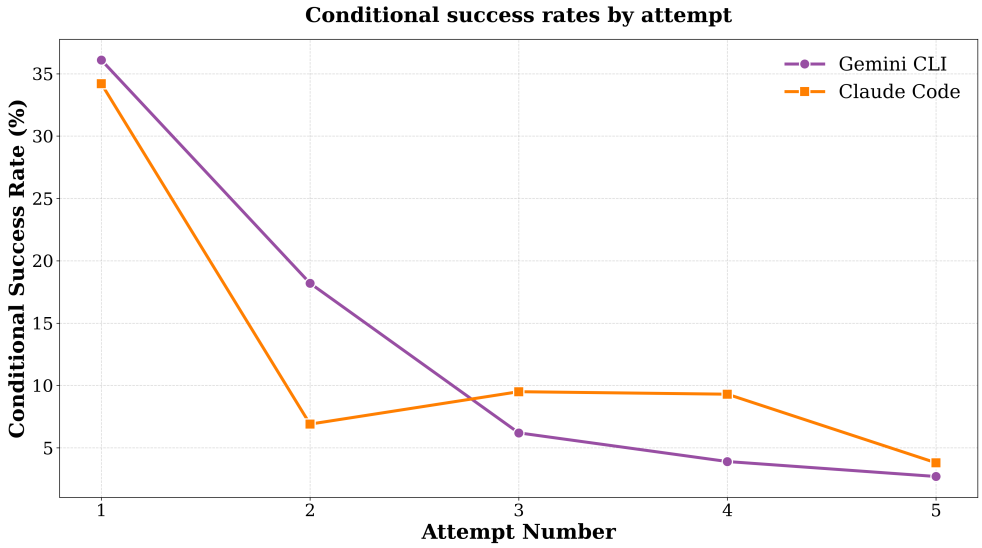


Fig. 3. Conditional success rates by attempt

Figure 3 demonstrates that the conditional success rates of both agents decline after the first attempt, though the patterns differ notably. Gemini CLI exhibits a sharp, monotonic decline from 36.1% on attempt 1 to 2.7% by attempt 5, suggesting progressively diminishing returns with each additional attempt. Claude Code, in contrast, shows a steeper initial drop from 34.2% to 6.9% between attempts 1 and 2, but then stabilizes at approximately 9% for attempts 3 and 4 before declining to 3.8% on the final attempt.

Claude Code demonstrated a substantial speed advantage in Table 4. Across all tests, Claude Code averaged 307.0 seconds (5.1 minutes) compared to Gemini CLI's 520.4 seconds (8.7 minutes), corresponding to a 41% reduction in execution time. For repairs, Claude Code completed in an average of 163.8 seconds (2.7 minutes) versus Gemini CLI's 281.6 seconds (4.7 minutes), making it 42% faster. These execution times corresponded to a throughput of approximately 11.7 tests per hour for Claude Code, compared to 6.9 for Gemini CLI.

Table 4. Execution time analysis

Metric	Gemini CLI	Claude Code
Successful Fixes		
Mean time	281.6s (4.7m)	163.8s (2.7m)
Median time	183.2s (3.1m)	101.9s (1.7m)
Failed Fixes		
Mean time	802.9s (13.4m)	459.8s (7.7m)
Median time	699.3s (11.7m)	334.3s (5.6m)
All Tests		
Mean time	520.4s (8.7m)	307.0s (5.1m)
Median time	440.8s (7.3m)	255.2s (4.3m)

Discussion The first-attempt dominance pattern, accounting for approximately 67% of repairs for both agents, indicates that when agents correctly diagnose the root cause from pre-validation results, they often implement an effective fix immediately. For tests not resolved on the first attempt, the divergent iteration patterns reveal distinct agent behaviors. Gemini CLI's monotonic decline, falling to a 2.7% success rate by the fifth attempt, suggests that its deliberate approach either succeeds initially or fails to identify viable alternatives. In contrast, Claude Code's success rate stabilizes at approximately 9% during attempts 3 and 4 before declining. This stabilization indicates that its faster iteration style may enable the exploration of alternative repair strategies that occasionally succeed. These findings suggest that speed-accuracy trade-offs in agent architecture influence not only throughput but also the capacity to recover from initial failures, though this effect remains modest given the overall low success rates observed beyond the second attempt.

While each attempt was prompted within the same thread to maintain context, agents struggled to generate substantially different repair strategies after their initial approaches failed. This could have indicated:

- **Test Difficulty**: Tests requiring multiple attempts involved complex root causes (e.g., subtle cross-test dependencies) that were inherently harder to diagnose and fix.
- **Strategy Exhaustion**: After attempting common fix patterns (fixture isolation, state reset), agents had limited alternative strategies to explore.
- **Broken Repository Setup**: Failures may have stemmed from incorrect or incomplete repository setup, where the environment initialization did not execute the code under test correctly, preventing meaningful repair attempts.

Execution analysis revealed substantial practical differences between the agents. Claude Code's 41–42% faster execution delivers nearly 70% higher throughput (11.7 vs. 6.9 tests per hour), making it significantly more cost-effective for large-scale deployment. Notably, Claude Code achieved this speed advantage while maintaining nearly identical success rates (51.6% vs. 54.2%), suggesting its performance reflects more efficient processing rather than reduced thoroughness. This consistent advantage across both successful repairs (42% faster) and all tests (41% faster) demonstrates that Claude Code's efficiency remains stable regardless of outcome, indicating architectural differences in how agents allocate computational resources during diagnosis and implementation.

The considerably longer execution times for failed attempts reflected the five-attempt exhaustion pattern and reinforced that iteration budgets beyond the first attempt provided diminishing value. These findings suggest that future improvements should focus on enhancing first-attempt diagnosis through better prompts and root cause analysis rather than increasing iteration limits.

6 Threats to Validity

Internal Validity Internal validity concerns factors that may affect the accuracy and reliability of our results within the scope of this study.

- **Prompt Sensitivity**: Our results depend on a specific prompt design (Section 4.3). Alternative prompt structures, different examples, or varied constraint specifications could yield different success rates. While we designed the prompt based on pilot testing, we did not systematically evaluate prompt variations, meaning our results reflect a single point in the prompt engineering design space.
- **Non-Determinism in LLM Responses**: LLMs are inherently non-deterministic, so the same prompt can produce different outputs across runs. As a result, success rates may differ if the experiments are repeated, and a test classified as failed could succeed in another run

due to different random choices. This also means that comparisons between models may be imperfect if one model benefited from favorable random choices on specific tests.
- **Limited Validation Runs**: Our validation procedure relies on a limited number of isolated executions and a bounded set of randomized test-order permutations, which constrains the ability to detect rare flakiness patterns. Tests that fail only under a small fraction of execution orderings may not manifest failures within a limited sample, leading to false positives where a test is incorrectly classified as fixed despite remaining non-deterministic. Consequently, the reported success rates can underestimate the true difficulty of achieving robust fixes, and some repairs may be overfitted to the specific validation sequences rather than generalizing to all possible execution orderings.

External Validity External validity concerns factors that may limit the generalizability of our findings to other contexts, languages, or test suites.

- **Single-Language Scope**: Our implementation targeted Python with pytest exclusively. This specificity introduced two potential biases. First, Python's characteristics, such as dynamic typing, global interpreter lock limiting true concurrency, and widespread pytest adoption, may have made certain flakiness patterns easier to fix than in statically-typed or concurrent languages. Second, Python's dominance in LLM training data [35] may have inflated performance relative to less-represented languages, where models have less exposure to idiomatic testing patterns. Thus, the reported success rates (51–54%) should be interpreted as potentially optimistic when considering application to diverse language ecosystems.
- **Dataset Constraints**: The IDoFT dataset, while valuable for studying flaky test repair in real-world projects, has limitations that should be considered when interpreting the results. Our evaluation was limited to a portion of IDoFT tests where we could successfully reproduce the testing environment. This introduces systematic bias favoring tests with simpler dependencies, smaller codebases, and better maintenance practices. The remaining tests excluded due to setup failures may represent more challenging flakiness patterns. Consequently, our measured success rate likely represents an upper bound on current CLI coding agent capabilities when applied to real-world test suites with full complexity and maintenance status diversity.
- **Infrastructure Context**: Tests that interacted with databases, external APIs, or file systems were particularly challenging. While LLMs may have recognized the need for isolation or mocking, they often struggled to implement it correctly due to missing infrastructure context, incomplete knowledge of API contracts, or implicit environmental assumptions. This meant that tests requiring complex external system interactions may have been systematically underrepresented in our reported success rates.

7 Future Research

- **Cross-Language Generalization** The IDoFT dataset includes substantial Java repositories, enabling investigation of cross-language transfer. Comparative evaluation across Python and Java would reveal whether observed success patterns (e.g., first-attempt dominance) generalize or reflect language-specific characteristics. Extension to statically-typed languages could also test whether static type systems aid or hinder autonomous repair.
- **Ensemble and Hybrid Approaches** Our results showed that Gemini CLI and Claude Code exhibited complementary strengths, with 18 tests (11.6%) fixed exclusively by one agent. Future work could explore ensemble strategies where multiple agents attempt each test, with a voting or confidence-based selection mechanism choosing the best fix. Hybrid approaches combining fast initial attempts (Claude Code) with thorough secondary attempts (Gemini CLI) could optimize both speed and success rate, potentially increasing the overall success rate.

– **Human Performance Baseline** A critical missing baseline is human developer performance on the same task. A controlled study in which developers repair the same test set would enable comparison of success rates, time to fix, and qualitative repair strategies. Such comparison would contextualize whether 51–54% success in 5–9 minutes represents strong or weak performance relative to human capabilities, informing deployment expectations.

8 Conclusion

This paper presented the first systematic evaluation of CLI coding agents for autonomous order-dependent flaky test repair, comparing Gemini CLI and Claude Code on real-world Python projects. Our evaluation confirms that CLI coding agents are practically viable for autonomous flaky test repair, achieving overall success rates exceeding 50% without specialized fine-tuning. However, performance is highly dependent on the flakiness category, with agents repairing nearly 70% of victim tests but only about 23% of brittle tests. This substantial gap underscores the critical role of preliminary test pass and fail information, as providing only failure-inducing sequences prevents agents from effectively identifying the missing state dependencies needed to repair brittle tests.

Our results show that the first repair attempt is crucial, accounting for the majority of successful fixes. Because agent performance degrades rapidly in subsequent iterations, we find that a budget of three attempts is sufficient to capture actionable repairs while avoiding wasted resources. Although both agents achieve similar success rates, Claude Code proves significantly more efficient, operating over 40% faster than Gemini CLI, which positions it as the more scalable option for integration into real-world development pipelines.

References

1. Akli, A., Haben, G., Habchi, S., Papadakis, M., Le Traon, Y.: Flakycat: Predicting flaky tests categories using few-shot learning. In: FlakyCat: Predicting Flaky Tests Categories using Few-Shot Learning (May 2023). https://doi.org/10.1109/AST58925.2023.00018
2. Bell, J., Legunsen, O., Hilton, M., Eloussi, L., Yung, T., Marinov, D.: Deflaker: automatically detecting flaky tests. In: Proceedings of the 40th International Conference on Software Engineering. p. 433–444. ICSE '18, Association for Computing Machinery, New York, NY, USA (2018). https://doi.org/10.1145/3180155.3180164
3. Bogavelli, T., Sharma, R., Subramani, H.: Agentarch: A comprehensive benchmark to evaluate agent architectures in enterprise (2026), `https://arxiv.org/abs/2509.10769`
4. Bouzenia, I., Devanbu, P., Pradel, M.: Repairagent: An autonomous, llm-based agent for program repair (2024), `https://arxiv.org/abs/2403.17134`
5. Chen, Y., Jabbarvand, R.: A generic approach to fix test flakiness in real-world projects (2024), `https://arxiv.org/abs/2404.09398`
6. Chen, Y., Jabbarvand, R.: Neurosymbolic repair of test flakiness. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. p. 1402–1414. ISSTA 2024, Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3650212.3680369
7. Dong, Y., Jiang, X., Qian, J., Wang, T., Zhang, K., Jin, Z., Li, G.: A survey on code generation with llm-based agents (2025), `https://arxiv.org/abs/2508.00083`
8. Eck, M., Palomba, F., Castelluccio, M., Bacchelli, A.: Understanding flaky tests: the developer's perspective. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. p. 830–840. ESEC/FSE 2019, Association for Computing Machinery, New York, NY, USA (2019). https://doi.org/10.1145/3338906.3338945
9. Fatima, S., Hemmati, H., C. Briand, L.: Flakyfix: Using large language models for predicting flaky test fix categories and test code repair. IEEE Transactions on Software Engineering **50**(12), 3146–3171 (2024). https://doi.org/10.1109/TSE.2024.3472476

10. Gambi, A., Bell, J., Zeller, A.: Practical test dependency detection. In: Proceedings - 2018 IEEE 11th International Conference on Software Testing, Verification and Validation, ICST 2018. pp. 1–11. Proceedings - 2018 IEEE 11th International Conference on Software Testing, Verification and Validation, ICST 2018, IEEE Computer Society (May 2018). https://doi.org/10.1109/ICST.2018.00011
11. Gruber, M., Fraser, G.: A survey on how test flakiness affects developers and what support they need to address it. In: 2022 IEEE Conference on Software Testing, Verification and Validation (ICST). pp. 82–92 (2022). https://doi.org/10.1109/ICST53961.2022.00020
12. Gruber, M., Lukasczyk, S., Kroiß, F., Fraser, G.: An empirical study of flaky tests in python. In: 2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST). pp. 148–158 (2021). https://doi.org/10.1109/ICST49551.2021.00026
13. Hu, X., Niu, F., Chen, J., Zhou, X., Zhang, J., He, J., Xia, X., Lo, D.: Assessing and advancing benchmarks for evaluating large language models in software engineering tasks. ACM Trans. Softw. Eng. Methodol. (Dec 2026). https://doi.org/10.1145/3786771, just Accepted
14. Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.R.: SWE-bench: Can language models resolve real-world github issues? In: The Twelfth International Conference on Learning Representations (2024), https://openreview.net/forum?id=VTF8yNQM66
15. King, T.M., Santiago, D., Phillips, J., Clarke, P.J.: Towards a bayesian network model for predicting flaky automated tests. In: 2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C). pp. 100–107 (2018). https://doi.org/10.1109/QRS-C.2018.00031
16. Lam, W.: International Dataset of Flaky Tests (IDoFT) (2020), http://mir.cs.illinois.edu/flakytests
17. Lam, W., Muslu, K., Sajnani, H., Thummalapenta, S.: A study on the lifecycle of flaky tests. In: ICSE 2020 (July 2020), https://www.microsoft.com/en-us/research/publication/a-study-on-the-lifecycle-of-flaky-tests/
18. Lam, W., Oei, R., Shi, A., Marinov, D., Xie, T.: iDFlakies: A framework for detecting and partially classifying flaky tests. In: ICST 2019: 12th IEEE International Conference on Software Testing, Verification and Validation. pp. 312–322. Xi'an, China (April 2019). https://doi.org/10.1109/ICST.2019.00038
19. Li, C., Behrang, F., Shi, A., Liu, P.: Flakyguard: Automatically fixing flaky tests at industry scale (2025), https://arxiv.org/abs/2511.14002
20. Li, C., Zhu, C., Wang, W., Shi, A.: Repairing order-dependent flaky tests via test generation. In: Proceedings of the 44th International Conference on Software Engineering. p. 1881–1892. ICSE '22, Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3510003.3510173
21. Li, H., Zhang, H., Hassan, A.E.: The rise of ai teammates in software engineering (se) 3.0: How autonomous coding agents are reshaping software engineering (2025), https://arxiv.org/abs/2507.15003
22. Liang, J.T., Yang, C., Myers, B.A.: A large-scale survey on the usability of ai programming assistants: Successes and challenges. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24, Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3597503.3608128
23. Liu, J., Wang, K., Chen, Y., Peng, X., Chen, Z., Zhang, L., Lou, Y.: Large language model-based agents for software engineering: A survey (2025), https://arxiv.org/abs/2409.02977
24. Luo, Q., Hariri, F., Eloussi, L., Marinov, D.: An empirical analysis of flaky tests. In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering. p. 643–653. FSE 2014, Association for Computing Machinery, New York, NY, USA (2014). https://doi.org/10.1145/2635868.2635920
25. Luo, W., Keung, J.W., Yang, B., Klein, J., Bissyande, T.F., Tian, H., Le, B.: Unlocking llm repair capabilities through cross-language translation and multi-agent refinement (2025), https://arxiv.org/abs/2503.22512
26. More, R., Bradbury, J.S.: An analysis of llm fine-tuning and few-shot learning for flaky test detection and classification (2025), https://arxiv.org/abs/2502.02715
27. Mündler, N., Mueller, M.N., He, J., Vechev, M.: SWT-bench: Testing and validating real-world bug-fixes with code agents. In: The Thirty-eighth Annual Conference on Neural Information Processing Systems (2024), https://openreview.net/forum?id=9Y8zUO11EQ

28. Palomba, F., Zaidman, A.: Notice of retraction: Does refactoring of test smells induce fixing flaky tests? In: 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME). pp. 1–12 (2017). https://doi.org/10.1109/ICSME.2017.12
29. Parry, O., Kapfhammer, G.M., Hilton, M., McMinn, P.: Surveying the developer experience of flaky tests. In: Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice. p. 253–262. ICSE-SEIP '22, Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3510457.3513037
30. Rondon, P., Wei, R., Cambronero, J., Cito, J., Sun, A., Sanyam, S., Tufano, M., Chandra, S.: Evaluating agent-based program repair at google (2025), https://arxiv.org/abs/2501.07531
31. Sergeyuk, A., Golubev, Y., Bryksin, T., Ahmed, I.: Using ai-based coding assistants in practice: State of affairs, perceptions, and ways forward. Information and Software Technology **178**, 107610 (2025). https://doi.org/10.1016/j.infsof.2024.107610, https://www.sciencedirect.com/science/article/pii/S0950584924002155
32. Shi, A., Lam, W., Oei, R., Xie, T., Marinov, D.: ifixflakies: a framework for automatically fixing order-dependent flaky tests. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. p. 545–555. ESEC/FSE 2019, Association for Computing Machinery, New York, NY, USA (2019). https://doi.org/10.1145/3338906.3338925
33. Shi, A., Lam, W., Oei, R., Xie, T., Marinov, D.: ifixflakies: a framework for automatically fixing order-dependent flaky tests. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. p. 545–555. ESEC/FSE 2019, Association for Computing Machinery, New York, NY, USA (2019). https://doi.org/10.1145/3338906.3338925
34. Silva, D., Teixeira, L., d'Amorim, M.: Shake it! detecting flaky tests caused by concurrency with shaker. In: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME). pp. 301–311 (2020). https://doi.org/10.1109/ICSME46990.2020.00037
35. Twist, L., Zhang, J.M., Harman, M., Syme, D., Noppen, J., Yannakoudakis, H., Nauck, D.: A study of llms' preferences for libraries and programming languages (2025), https://arxiv.org/abs/2503.17181
36. Wang, H., Gong, J., Zhang, H., Xu, J., Wang, Z.: Ai agentic programming: A survey of techniques, challenges, and opportunities (2025), https://arxiv.org/abs/2508.11126
37. Wang, N., Hu, X., Liu, P., Zhu, H., Hou, Y., Huang, H., Zhang, S., Yang, J., Liu, J., Zhang, G., Zhang, C., Wang, J., Jiang, Y.E., Zhou, W.: Efficient agents: Building effective agents while reducing cost (2025), https://arxiv.org/abs/2508.02694
38. Wang, R., Chen, Y., Lam, W.: ipflakies: a framework for detecting and fixing python order-dependent flaky tests. In: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings. p. 120–124. ICSE '22, Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3510454.3516846
39. White, C., Dooley, S., Roberts, M., Pal, A., Feuer, B., Jain, S., Shwartz-Ziv, R., Jain, N., Saifullah, K., Dey, S., Shubh-Agrawal, Sandha, S.S., Naidu, S.V., Hegde, C., LeCun, Y., Goldstein, T., Neiswanger, W., Goldblum, M.: Livebench: A challenging, contamination-free LLM benchmark. In: The Thirteenth International Conference on Learning Representations (2025)
40. Xie, L., Li, Z., Pei, Y., Wen, Z., Liu, K., Zhang, T., Li, X.: Premm: Llm-based program repair for multi-method bugs via divide and conquer. Proc. ACM Program. Lang. **9**(OOPSLA2) (Oct 2025). https://doi.org/10.1145/3763097
41. Zhang, S., Jalali, D., Wuttke, J., Muşlu, K., Lam, W., Ernst, M.D., Notkin, D.: Empirically revisiting the test independence assumption. In: Proceedings of the 2014 International Symposium on Software Testing and Analysis. p. 385–396. ISSTA 2014, Association for Computing Machinery, New York, NY, USA (2014). https://doi.org/10.1145/2610384.2610404
42. Ziftci, C., Cavalcanti, D.: De-flake your tests : Automatically locating root causes of flaky tests in code at google. In: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME). pp. 736–745 (2020). https://doi.org/10.1109/ICSME46990.2020.00083
43. Zolfaghari, B., Parizi, R.M., Srivastava, G., Hailemariam, Y.: Root causing, detecting, and fixing flaky tests: State of the art and future roadmap. Software: Practice and Experience **51**(5), 851–867 (2021). https://doi.org/10.1002/spe.2929, https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.2929